

Cuestionario Tecnico de Control:
Recursividad, Arreglos y Punteros en Memoria RAM

Sofia Mojica, Gabriel Zacarias, Mateo Roberts y Leonel De Armas
Escuela de Educacion Secundaria Tecnica N.º 1 (EEST N.º 1)
Lenguaje de Programacion (LPR2026)
Actividad N.º 6 – Proyecto Integrador de Area (P.I.A.) – [P3_HQI_2.0]

Profesores: Elias York – Yamil Ganduglia

2026

Integrantes del Grupo y Distribucion de Tareas

El presente trabajo fue elaborado en forma grupal (P.I.A.) por cuatro integrantes. A continuacion se detalla el nombre de cada integrante y la tarea especifica que realizo dentro del informe:

Integrante	Tarea especifica realizada
Sofia Mojica	Investigacion y redaccion de la Pregunta 1 (Caso Base en recursividad); revision de ortografia, coherencia y normas APA v7 de todo el documento.
Gabriel Zacarias	Investigacion y redaccion de la Pregunta 2 (Eficiencia de la instruccion break); verificacion practica del comportamiento del bucle mediante pruebas de codigo en el compilador.
Mateo Roberts	Investigacion y redaccion de la Pregunta 3 (Direccionamiento y operador &); recopilacion y organizacion de las referencias bibliograficas en formato APA.
Leonel De Armas	Investigacion y redaccion de la Pregunta 4 (Variable temporal en el intercambio de punteros); armado final del documento, portada y exportacion del archivo PDF.

Introduccion

El siguiente informe tiene como objetivo responder el Cuestionario Tecnico de Control correspondiente a la Actividad N.º 6, abordando cuatro problemáticas centrales del funcionamiento de un programa a bajo nivel: el comportamiento de la memoria RAM ante una recursividad sin caso base, la optimización que aporta la instrucción break durante una búsqueda en arreglos, la comprobación de la contigüidad de celdas de memoria mediante el operador de dirección (&), y el rol de las variables temporales en el intercambio de valores mediante punteros. Finalmente, se incluye una autoevaluación grupal sobre el reto que resultó más complejo de comprender a bajo nivel.

Pregunta 1: Caso Base – Error crítico en la memoria RAM sin caso base de corte

Una función recursiva que carece de un caso base de corte nunca detiene su cadena de autollamadas. Cada vez que la función se invoca a sí misma, el sistema operativo reserva un nuevo marco de pila (stack frame) dentro del segmento de pila (stack) de la memoria RAM, donde se almacenan las variables locales, los parámetros y la dirección de retorno de esa llamada en particular. Como la función jamás alcanza una condición que le indique retornar, estos marcos se siguen apilando uno sobre otro de manera indefinida.

Dado que el segmento de pila tiene un tamaño finito y mucho menor que el de la memoria total del sistema, este proceso agota rápidamente el espacio disponible. El error crítico que se produce se conoce como desbordamiento de pila o stack overflow: el programa intenta escribir un nuevo marco en una dirección de memoria que ya está fuera del área reservada para la pila, invadiendo zonas de memoria que no le corresponden. El sistema operativo detecta esta violación de acceso y finaliza el programa de forma abrupta, generalmente mostrando una excepción o un fallo de segmentación (segmentation fault), ya que no queda memoria disponible para continuar apilando nuevas llamadas ni para que estas retornen ordenadamente hacia la función que las originó.

Pregunta 2: Eficiencia – Optimizacion del rendimiento mediante la instruccion break

Cuando se realiza una busqueda secuencial dentro de un arreglo, el bucle recorre cada elemento comparandolo con el valor buscado. Si no se utiliza la instruccion break, el bucle continua ejecutando iteraciones y comparaciones hasta llegar al ultimo indice del arreglo, incluso despues de haber encontrado el elemento deseado. Esto implica que el microprocesador sigue realizando ciclos de reloj, incrementos del contador, comparaciones y accesos a memoria de forma totalmente innecesaria.

La instruccion break interrumpe la ejecucion del bucle en el instante exacto en que se cumple la condicion de busqueda, evitando que el procesador siga procesando los elementos restantes del arreglo. En terminos de complejidad algoritmica, esto reduce el numero de operaciones realizadas en el caso promedio y en el mejor caso, ya que el tiempo de ejecucion pasa a depender de la posicion en la que se encuentra el dato y no siempre del tamano total del arreglo. Menos iteraciones significan menos ciclos de reloj consumidos, menos accesos a la memoria cache y, en consecuencia, un menor tiempo de respuesta y un menor consumo energetico del microprocesador durante la ejecucion de la busqueda.

Pregunta 3: Direccionamiento – Comprobacion de contiguidad mediante el operador &

El operador de direccion (&) permite obtener la direccion de memoria en la que se encuentra almacenada una variable o un elemento de un arreglo. Para comprobar que los elementos de un array ocupan celdas contiguas en la memoria RAM, basta con imprimir la direccion de cada elemento aplicando el operador & sobre cada posicion del arreglo, por ejemplo &arreglo[0], &arreglo[1], &arreglo[2], y asi sucesivamente, generalmente mostrando el resultado en formato hexadecimal.

Al comparar los valores obtenidos, se observa que la diferencia entre la direccion de un elemento y la del siguiente es siempre igual al tamano en bytes que ocupa el tipo de dato del arreglo (por ejemplo, 4 bytes para un arreglo de tipo int, o 1 byte para un arreglo de tipo char).

Esta diferencia constante y creciente demuestra que el compilador reservo un bloque unico e ininterrumpido de memoria para todo el arreglo, y que cada elemento se ubica inmediatamente despues del anterior. Es precisamente esta contiguidad la que permite calcular la direccion de cualquier elemento a partir de la direccion base del arreglo mas el indice multiplicado por el tamano del tipo de dato.

Pregunta 4: Variables Temporales – Funcion en el intercambio de valores con punteros

En un algoritmo de intercambio (swap) que utiliza punteros, la funcion recibe las direcciones de memoria de las dos variables a intercambiar en lugar de sus valores. El problema es que, si se intenta asignar directamente el contenido de una variable a la otra (por ejemplo, `*a = *b`), el valor original de la primera variable se pierde antes de poder ser copiado a la segunda, ya que ambas asignaciones no pueden ocurrir de manera simultanea.

Alli es donde interviene la variable temporal: su funcion es actuar como un espacio de almacenamiento intermedio que guarda copia del valor que esta a punto de ser sobrescrito. El procedimiento tipico es `temp = *a`; luego `*a = *b`; y finalmente `*b = temp`. De esta manera, la variable temporal conserva el valor original de la primera celda de memoria mientras esta es sobrescrita con el valor de la segunda, y luego permite restituir ese valor guardado en la segunda celda. Sin esta variable auxiliar, uno de los dos valores se perderia de forma irrecuperable durante el proceso de intercambio.

Pregunta 5: Autoevaluacion – Reto mas complejo y forma de resolucion

A continuacion, cada integrante del grupo expone de manera individual cual de los retos abordados (recursividad sin caso base, direccionamiento de arreglos o intercambio de valores con punteros) le resulto mas complejo de comprender a bajo nivel, y de que manera logro resolverlo:

Sofia Mojica: El reto que me resulto mas complejo fue entender el desbordamiento de pila en la recursividad, porque no lograba visualizar como se apilaban las llamadas en la memoria. Lo resolví dibujando en papel cada marco de pila con sus variables locales a medida que la función se llamaba a si misma, lo que me permitio ver claramente por que el programa se detenía de forma abrupta al no existir un caso base.

Gabriel Zacarias: Para mi, el punto mas difícil fue comprender por que break realmente mejora el rendimiento y no solo detiene el bucle. Lo resolví ejecutando el mismo algoritmo de búsqueda con y sin break, contando manualmente el número de iteraciones que realizaba el procesador en cada caso, lo cual me permitio comprobar la diferencia de eficiencia de forma practica.

Mateo Roberts: El desafío que mas trabajo me llevo fue el direccionamiento de arreglos, ya que al principio no entendia por que las direcciones de memoria aumentaban de a un número fijo. Logre resolverlo imprimiendo las direcciones de cada elemento del arreglo en el compilador y comparandolas manualmente con el tamaño del tipo de dato utilizado.

Leonel De Armas: Lo que mas me costo comprender fue el intercambio de valores mediante punteros, porque confundia el puntero con el valor al que apuntaba. Lo pude resolver haciendo un seguimiento paso a paso del algoritmo, escribiendo en cada línea el contenido de la variable temporal y de las celdas de memoria involucradas hasta entender el orden correcto de las asignaciones.

Conclusion

El desarrollo de este cuestionario permitio al grupo comprender que, por debajo de la sintaxis de un lenguaje de programación, existen mecanismos concretos de administración de memoria RAM que determinan el correcto o incorrecto funcionamiento de un programa. Comprender el caso base en la recursividad, el impacto de la instrucción break en el rendimiento, la contigüidad de los arreglos en memoria y el rol de las variables temporales en

el manejo de punteros resulta fundamental para escribir código eficiente, seguro y libre de errores críticos como el desbordamiento de pila o la pérdida de datos durante un intercambio de valores.

Referencias

Deitel, P., & Deitel, H. (2016). *Como programar en C* (8.^a ed.). Pearson Educacion.

Joyanes Aguilar, L. (2018). *Fundamentos de programacion: Algoritmos, estructuras de datos y objetos* (5.^a ed.). McGraw-Hill.

Kernighan, B. W., & Ritchie, D. M. (1991). *El lenguaje de programacion C* (2.^a ed.). Prentice Hall.

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.^a ed.).